

```

        0, 10, 0,
        10, 10, 0,
        0, 0, 10,
        10, 0, 10,
        0, 10, 10,
        10, 10, 10,
    };

    // convert vertex data to a byte array
    byte[] vertexData = vertices.SelectMany(v =>
    BitConverter.GetBytes(v)).ToArray();

    // write the locations to each vertex
    await stream.WriteAsync(vertexData, 0, vertexData.Length);
}
// update vertex count: 8 vertices in the cube
mesh.VertexCount = 8;
}

```

This next method defines all of the triangles to be drawn across these vertices:

C#

```

private static async Task SetTriangleIndicesAsync(Printing3DMesh mesh) {

    Printing3DBufferDescription description;

    description.Format = Printing3DBufferFormat.Printing3DUInt;
    // 3 vertex indices
    description.Stride = 3;
    // 12 triangles in all in the cube
    mesh.IndexCount = 12;

    mesh.TriangleIndicesDescription = description;

    // allocate space for 12 triangles
    mesh.CreateTriangleIndices(sizeof(UInt32) * 3 * 12);

    // get a datastream of the triangle indices (should be blank at this
    point)
    var stream2 = mesh.GetTriangleIndices().AsStream();
    {
        // define a set of triangle indices: each row is one triangle. The
        values in each row
        // correspond to the index of the vertex.
        UInt32[] indices =
        {
            1, 0, 2,
            1, 2, 3,
            0, 1, 5,
            0, 5, 4,
            1, 3, 7,
            1, 7, 5,

```

```

        2, 7, 3,
        2, 6, 7,
        0, 6, 2,
        0, 4, 6,
        6, 5, 7,
        4, 5, 6,
    };
    // convert index data to byte array
    var vertexData = indices.SelectMany(v =>
    BitConverter.GetBytes(v)).ToArray();
    var len = vertexData.Length;
    // write index data to the triangle indices stream
    await stream2.WriteAsync(vertexData, 0, vertexData.Length);
}

}

```

ⓘ Note

All triangles must have their indices defined in counter-clockwise order (when viewing the triangle from outside of the mesh object), so that their face-normal vectors point outward.

When a `Printing3DMesh` object contains valid sets of vertices and triangles, it should then be added to the **`Meshes`** property of the model. All **`Printing3DMesh`** objects in a package must be stored under the **`Meshes`** property of the **`Printing3DModel`** class, as shown here.

C#

```

// add the mesh to the model
model.Meshes.Add(mesh);

```

Create materials

A 3D model can hold data for multiple materials. This convention is intended to take advantage of 3D manufacturing devices that can use multiple materials on a single print job. There are also multiple *types* of material groups, each capable of supporting a number of different individual materials.

Each material group must have a unique reference ID number, and each material within that group must also have a unique ID. The different mesh objects within a model can then reference the materials.

Furthermore, individual triangles on each mesh can specify different materials and different materials can even be represented within a single triangle, with each triangle vertex having a different material assigned to it and the face material calculated as the gradient between them.

First we'll show how to create different kinds of materials within their respective material groups and store them as resources on the model object. Then, we'll assign different materials to individual meshes and individual triangles.

Base materials

The default material type is **Base Material**, which has both a **Color Material** value (described below) and a name attribute that is intended to specify the *type* of material to use.

C#

```
// add material group
// all material indices need to start from 1: 0 is a reserved id
// create new base materialgroup with id = 1
var baseMaterialGroup = new Printing3DBaseMaterialGroup(1);

// create color objects
// 'A' should be 255 if alpha = 100%
var darkBlue = Windows.UI.Color.FromArgb(255, 20, 20, 90);
var orange = Windows.UI.Color.FromArgb(255, 250, 120, 45);
var teal = Windows.UI.Color.FromArgb(255, 1, 250, 200);

// create new ColorMaterials, assigning color objects
var colrMat = new Printing3DColorMaterial();
colrMat.Color = darkBlue;

var colrMat2 = new Printing3DColorMaterial();
colrMat2.Color = orange;

var colrMat3 = new Printing3DColorMaterial();
colrMat3.Color = teal;

// setup new materials using the ColorMaterial objects
// set desired material type in the Name property
var baseMaterial = new Printing3DBaseMaterial {
    Name = Printing3DBaseMaterial.Pla,
    Color = colrMat
};

var baseMaterial2 = new Printing3DBaseMaterial {
    Name = Printing3DBaseMaterial.Abs,
    Color = colrMat2
};
```

```
// add base materials to the basematerialgroup

// material group index 0
baseMaterialGroup.Bases.Add(baseMaterial);
// material group index 1
baseMaterialGroup.Bases.Add(baseMaterial2);

// add material group to the basegroups property of the model
model.Material.BaseGroups.Add(baseMaterialGroup);
```

⚠ Note

The 3D manufacturing device will determine which available physical materials map to which virtual material elements stored in the 3MF. Material mapping doesn't have to be 1:1. If a 3D printer only uses one material, it will print the whole model in that material, regardless of which objects or faces were assigned different materials.

Color materials

Color Materials are similar to **Base Materials**, but they do not contain a name. Thus, they give no instructions as to what type of material should be used by the machine. They hold only color data, and let the machine choose the material type (the machine might prompt the user to choose). In the following example, the `colrMat` objects from the previous method are used on their own.

C#

```
// add ColorMaterials to the Color Material Group (with id 2)
var colorGroup = new Printing3DColorMaterialGroup(2);

// add the previous ColorMaterial objects to this ColorMaterialGroup
colorGroup.Colors.Add(colrMat);
colorGroup.Colors.Add(colrMat2);
colorGroup.Colors.Add(colrMat3);

// add colorGroup to the ColorGroups property on the model
model.Material.ColorGroups.Add(colorGroup);
```

Composite materials

Composite Materials instruct the manufacturing device to use a uniform mixture of different **Base Materials**. Each **Composite Material Group** must reference exactly one **Base Material Group** from which to draw ingredients. Additionally, the **Base Materials**

within this group that are to be made available must be listed in a **Material Indices** list, which each **Composite Material** will reference when specifying the ratios (every **Composite Material** is a ratio of **Base Materials**).

C#

```
// CompositeGroups
// create new composite material group with id = 3
var compositeGroup = new Printing3DCompositeMaterialGroup(3);

// indices point to base materials in BaseMaterialGroup with id =1
compositeGroup.MaterialIndices.Add(0);
compositeGroup.MaterialIndices.Add(1);

// create new composite materials
var compMat = new Printing3DCompositeMaterial();
// fraction adds to 1.0
compMat.Values.Add(0.2); // .2 of first base material in BaseMaterialGroup 1
compMat.Values.Add(0.8); // .8 of second base material in BaseMaterialGroup 1

var compMat2 = new Printing3DCompositeMaterial();
// fraction adds to 1.0
compMat2.Values.Add(0.5);
compMat2.Values.Add(0.5);

var compMat3 = new Printing3DCompositeMaterial();
// fraction adds to 1.0
compMat3.Values.Add(0.8);
compMat3.Values.Add(0.2);

var compMat4 = new Printing3DCompositeMaterial();
// fraction adds to 1.0
compMat4.Values.Add(0.4);
compMat4.Values.Add(0.6);

// add composites to group
compositeGroup.Composites.Add(compMat);
compositeGroup.Composites.Add(compMat2);
compositeGroup.Composites.Add(compMat3);
compositeGroup.Composites.Add(compMat4);

// add group to model
model.Material.CompositeGroups.Add(compositeGroup);
```

Texture coordinate materials

3MF supports the use of 2D images to color the surfaces of 3D models. This way, the model can convey much more color data per triangle face (as opposed to having just

one color value per triangle vertex). Like **Color Materials**, texture coordinate materials only convey color data. To use a 2D texture, a texture resource must first be declared.

ⓘ Note

Texture data belongs to the 3MF Package itself, not to the model part within the package.

C#

```
// texture resource setup
Printing3DTextureResource texResource = new Printing3DTextureResource();
// name conveys the path within the 3MF document
texResource.Name = "/3D/Texture/msLogo.png";

// in this case, we reference texture data in the sample appx, convert it to
// an IRandomAccessStream, and assign it as the TextureData
Uri texUri = new Uri("ms-appx:///Assets/msLogo.png");
StorageFile file = await StorageFile.GetFileFromApplicationUriAsync(texUri);
IRandomAccessStreamWithContentType iRandomAccessStreamWithContentType =
await file.OpenReadAsync();
texResource.TextureData = iRandomAccessStreamWithContentType;
// add this texture resource to the 3MF Package
localPackage.Textures.Add(texResource);

// assign this texture resource to a Printing3DModelTexture
var modelTexture = new Printing3DModelTexture();
modelTexture.TextureResource = texResource;
```

Next, we must fill out **Texture3Coord Materials**. Each of these references a texture resource and specifies a particular point on the image (in UV coordinates).

C#

```
// texture2Coord Group
// create new Texture2CoordMaterialGroup with id = 4
var tex2CoordGroup = new Printing3DTexture2CoordMaterialGroup(4);

// create texture materials:
// set up four tex2coordmaterial objects with four (u,v) pairs,
// mapping to each corner of the image:

var tex2CoordMaterial = new Printing3DTexture2CoordMaterial();
tex2CoordMaterial.U = 0.0;
tex2CoordMaterial.V = 1.0;
tex2CoordGroup.Texture2Coords.Add(tex2CoordMaterial);

var tex2CoordMaterial2 = new Printing3DTexture2CoordMaterial();
tex2CoordMaterial2.U = 1.0;
```

```

tex2CoordMaterial2.V = 1.0;
tex2CoordGroup.Texture2Coords.Add(tex2CoordMaterial2);

var tex2CoordMaterial3 = new Printing3DTexture2CoordMaterial();
tex2CoordMaterial3.U = 0.0;
tex2CoordMaterial3.V = 0.0;
tex2CoordGroup.Texture2Coords.Add(tex2CoordMaterial3);

var tex2CoordMaterial4 = new Printing3DTexture2CoordMaterial();
tex2CoordMaterial4.U = 1.0;
tex2CoordMaterial4.V = 0.0;
tex2CoordGroup.Texture2Coords.Add(tex2CoordMaterial4);

// add our Printing3DModelTexture to the Texture property of the group
tex2CoordGroup.Texture = modelTexture;

// add metadata about the texture so that u,v values can be used
model.Metadata.Add("tex4", "/3D/Texture/msLogo.png");
// add group to groups on the model's material
model.Material.Texture2CoordGroups.Add(tex2CoordGroup);

```

Map materials to faces

In order to specify which materials are mapped to which vertices on each triangle, more work is required on the model's mesh object (if a model contains multiple meshes, they must each have their materials assigned separately). As mentioned above, materials are assigned per-vertex, per-triangle. The following example shows how this information is entered and interpreted.

C#

```

private static async Task SetMaterialIndicesAsync(Printing3DMesh mesh) {
    // declare a description of the material indices
    Printing3DBufferDescription description;
    description.Format = Printing3DBufferFormat.Printing3DUInt;
    // 4 indices for material description per triangle
    description.Stride = 4;
    // 12 triangles total
    mesh.IndexCount = 12;
    mesh.TriangleMaterialIndicesDescription = description;

    // create space for storing this data
    mesh.CreateTriangleMaterialIndices(sizeof(UInt32) * 4 * 12);

    {
        // each row is a triangle face (in the order they were created)
        // first column is the id of the material group, last 3 columns show
        // which material id (within that group)
        // maps to each triangle vertex (in the order they were listed when
        // creating triangles)
    }
}

```

```

        UInt32[] indices =
        {
            // base materials:
            // in the BaseMaterialGroup (id=1), the BaseMaterial with id=0
            will be applied to these triangle vertices
            1, 0, 0, 0,
            1, 0, 0, 0,
            // color materials:
            // in the ColorMaterialGroup (id=2), the ColorMaterials with
            these ids will be applied to these triangle vertices
            2, 1, 1, 1,
            2, 1, 1, 1,
            2, 0, 0, 0,
            2, 0, 0, 0,
            2, 0, 1, 2,
            2, 1, 0, 2,
            // composite materials:
            // in the CompositeMaterialGroup (id=3), the CompositeMaterial
            with id=0 will be applied to these triangles
            3,0,0,0,
            3,0,0,0,
            // texture materials:
            // in the Texture2CoordMaterialGroup (id=4), each texture
            coordinate is mapped to the appropriate vertex on these
            // two adjacent triangle faces, so that the square face they
            create displays the original rectangular image
            4, 0, 3, 1,
            4, 2, 3, 0,
        };

        // get the current (unassigned) vertex data as a stream and write
        our new 'indices' data to it.
        var stream = mesh.GetTriangleMaterialIndices().AsStream();
        var vertexData = indices.SelectMany(v =>
        BitConverter.GetBytes(v)).ToArray();
        var len = vertexData.Length;
        await stream.WriteAsync(vertexData, 0, vertexData.Length);
    }
}

```

Components and build

The component structure allows the user to place more than one mesh object in a printable 3D model. A [Printing3DComponent](#) object contains a single mesh and a list of references to other components. This is actually a list of

[Printing3DComponentWithMatrix](#) objects. [Printing3DComponentWithMatrix](#) objects each contain a [Printing3DComponent](#) and a transform matrix that applies to the mesh and contained components of the [Printing3DComponent](#).

For example, a model of a car might consist of a "Body" **Printing3DComponent** that holds the mesh for the car's body. The "Body" component may then contain references to four different **Printing3DComponentWithMatrix** objects, which all reference the same **Printing3DComponent** while the "Wheel" mesh may contain four different transform matrices (mapping the wheels to four different positions on the car's body). In this scenario, the "Body" mesh and "Wheel" mesh would each only need to be stored once, even though the final product would feature five meshes in total.

All **Printing3DComponent** objects must be directly referenced in the **Components** property of the model. The one particular component that is to be used in the printing job is stored in the **Build** Property.

C#

```
// create new component
Printing3DComponent component = new Printing3DComponent();

// assign mesh to the component's mesh
component.Mesh = mesh;

// add component to the model's list of all used components
// a model can have references to multiple components
model.Components.Add(component);

// create the transform matrix
var componentWithMatrix = new Printing3DComponentWithMatrix();
// assign component to this componentwithmatrix
componentWithMatrix.Component = component;

// create an identity matrix
var identityMatrix = Matrix4x4.Identity;

// use the identity matrix as the transform matrix (no transformation)
componentWithMatrix.Matrix = identityMatrix;

// add component to the build property.
model.Build.Components.Add(componentWithMatrix);
```

Save the package

Now that we have a model, with defined materials and components, we can save it to the package.

C#

```
// save the model to the package:
await localPackage.SaveModelToPackageAsync(model);
```

```
// get the model stream
var modelStream = localPackage.ModelPart;

// fix any textures in the model file
localPackage.ModelPart = await FixTextureContentType(modelStream);
```

The following function ensures the texture is specified correctly.

C#

```
/// <summary>
/// Ensure textures are saved correctly.
/// </summary>
/// <param name="modelStream">3dmodel.model data</param>
/// <returns></returns>
private async Task<IRandomAccessStream>
FixTextureContentType(IRandomAccessStream modelStream) {
    XDocument xmlDoc = XDocument.Load(modelStream.AsStreamForRead());

    var outputStream = new
Windows.Storage.Streams.InMemoryRandomAccessStream();
    var writer = new Windows.Storage.Streams.DataWriter(outputStream);
    writer.UnicodeEncoding = Windows.Storage.Streams.UnicodeEncoding.Utf8;
    writer.ByteOrder = Windows.Storage.Streams.ByteOrder.LittleEndian;
    writer.WriteString("<?xml version=\"1.0\" encoding=\"UTF-8\"?>");

    var text = xmlDoc.ToString();
    // ensure that content type is set correctly
    // texture content can be either png or jpg
    var replacedText = text.Replace("contenttype=\"\"",
"contenttype=\"image/png\"");
    writer.WriteString(replacedText);

    await writer.StoreAsync();
    await writer.FlushAsync();
    writer.DetachStream();
    return outputStream;
}
```

From here, we can either initiate a print job within the app (see [3D printing from your app](#)), or save this **Printing3D3MFPackage** as a .3mf file.

The following method takes a finished **Printing3D3MFPackage** and saves its data to a .3mf file.

C#

```
private async void SaveTo3mf(Printing3D3MFPackage localPackage) {

    // prompt the user to choose a location to save the file to
    FileSavePicker savePicker = new FileSavePicker();
```

```

savePicker.DefaultFileExtension = ".3mf";
savePicker.SuggestedStartLocation = PickerLocationId.DocumentsLibrary;
savePicker.FileTypeChoices.Add("3MF File", new[] { ".3mf" });
var storageFile = await savePicker.PickSaveFileAsync();
if (storageFile == null) {
    return;
}

// save the 3MF Package to an IRandomAccessStream
using (var stream = await localPackage.SaveAsync()) {
    // go to the beginning of the stream
    stream.Seek(0);

    // read from the file stream and write to a buffer
    using (var dataReader = new DataReader(stream)) {
        await dataReader.LoadAsync((uint)stream.Size);
        var buffer = dataReader.ReadBuffer((uint)stream.Size);

        // write from the buffer to the storagefile specified
        await FileIO.WriteBufferAsync(storageFile, buffer);
    }
}
}

```

Related topics

[3D printing from your app](#)

[3D printing UWP sample](#) 

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Devices, sensors, and power

Article • 05/06/2023

Integrate external devices or sensors into your Windows app to provide enhanced experiences for your users.

This section explains how to support various types of devices and sensors and add related features to your app, including enhanced print experiences, motion and orientation sensors, network connections, and battery status.

Create an NFC Smart Card app

Article • 06/13/2023

Important

This topic applies to Windows 10 Mobile only.

This topic describes how to use Host Card Emulation (HCE) to communicate directly with a near-field communication (NFC) card reader and let your customers access your services through their phone (instead of a physical card) without a mobile-network operator (MNO).

What you need to develop an HCE app

To develop an HCE-based card emulation app, you need to install Microsoft Visual Studio 2015 (see the [Visual Studio download page](#)) (includes the Windows developer tools) and the [Windows 10 Mobile emulator](#).

For more information about getting setup, see [Test with the Microsoft Emulator for Windows 10 Mobile](#).

Optionally, if you want to test with a real Windows 10 Mobile device instead of the included Windows 10 Mobile emulator, you will also need the following items.

- A Windows 10 Mobile device with NFC HCE support.
- A reader terminal that supports protocols ISO/IEC 14443-4 and ISO/IEC 7816-4

Windows 10 Mobile implements an HCE service that provides the following functionalities.

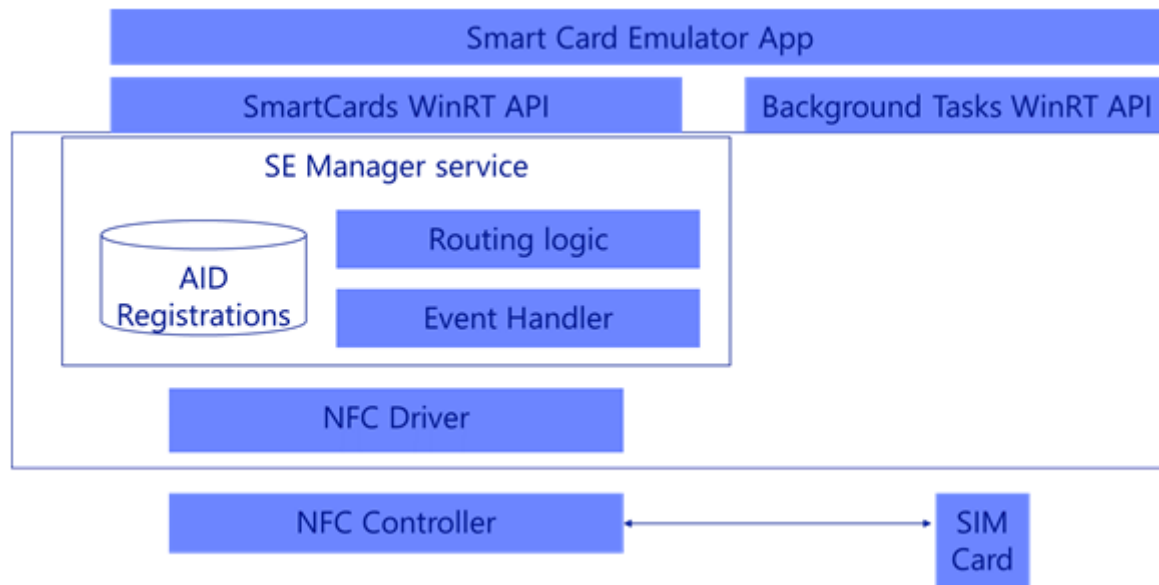
- Apps can register the applet identifiers (AIDs) for the cards they would like to emulate.
- Conflict resolution and routing of the Application Protocol Data Unit (APDU) command and response pairs to one of the registered apps based on the external reader card selection and user preference.
- Handling of events and notifications to the apps as a result of user actions.

Windows 10 supports emulation of smart cards that are based on ISO-DEP (ISO-IEC 14443-4) and communicates using APDUs as defined in the ISO-IEC 7816-4 specification. Windows 10 supports ISO/IEC 14443-4 Type A technology for HCE apps.

Type B, type F, and non-ISO-DEP (eg MIFARE) technologies are routed to the SIM by default.

Only Windows 10 Mobile devices are enabled with the card emulation feature. SIM-based and HCE-based card emulation is not available on other versions of Windows 10.

The architecture for HCE and SIM based card emulation support is shown in the diagram below.



App selection and AID routing

To develop an HCE app, you must understand how Windows 10 Mobile devices route AIDs to a specific app because users can install multiple different HCE apps. Each app can register multiple HCE and SIM-based cards.

When the user taps their Windows 10 Mobile device to a terminal, the data is automatically routed to the proper app installed on the device. This routing is based on the applet ID (AID) which is a 5-16 byte identifier. During a tap, the external terminal will transmit a SELECT command APDU to specify the AID it would like all subsequent APDU commands to be routed to. Subsequent SELECT commands, will change the routing again. Based on the AIDs registered by apps and user settings, the APDU traffic is routed to a specific app, which will send a response APDU. Be aware that a terminal may want to communicate with several different apps during the same tap. So you must ensure your app's background task exits as quickly as possible when deactivated to make room for another app's background task to respond to the APDU. We will discuss background tasks later in this topic.

HCE apps must register themselves with particular AIDs they can handle so they will receive APDUs for an AID. Apps declare AIDs by using AID groups. An AID group is

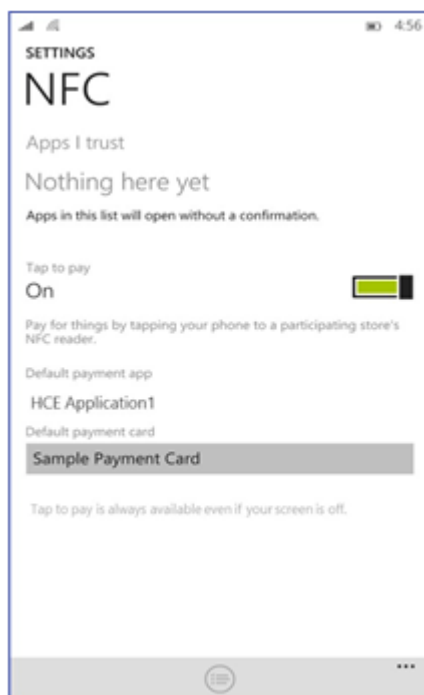
conceptually equivalent to an individual physical card. For example, one credit card is declared with an AID group and a second credit card from a different bank is declared with a different, second AID group, even though both of them may have the same AID.

Conflict resolution for payment AID groups

When an app registers physical cards (AID groups), it can declare the AID group category as either "Payment" or "Other." While there can be multiple payment AID groups registered at any given time, only one of these payment AID groups may be enabled for Tap and Pay at a time, which is selected by the user. This behavior exists because the user expects to be in control of consciously choosing a single payment, credit, or debit card to use so they don't pay with a different unintended card when tapping their device to a terminal.

However, multiple AID groups registered as "Other" can be enabled at the same time without user interaction. This behavior exists because other types of cards like loyalty, coupons, or transit are expected to just work without any effort or prompting whenever they tap their phone.

All the AID groups that are registered as "Payment" appear in the list of cards in the NFC Settings page, where the user can select their default payment card. When a default payment card is selected, the app that registered this payment AID group becomes the default payment app. Default payment apps can enable or disable any of their AID groups without user interaction. If the user declines the default payment app prompt, then the current default payment app (if any) continues to remain as default. The following screenshot shows the NFC Settings page.



Using the example screenshot above, if the user changes his default payment card to another card that is not registered by "HCE Application 1," the system creates a confirmation prompt for the user's consent. However, if the user changes his default payment card to another card that is registered by "HCE Application 1", the system does not create a confirmation prompt for the user as "HCE Application1" is already the default payment app.

Conflict resolution for non-payment AID groups

Non-payment cards categorized as "Other" do not appear in the NFC settings page.

Your app can create, register and enable non-payment AID groups in the same manner as payment AID groups. The main difference is that for non-payment AID groups the emulation category is set to "Other" as opposed to "Payment". After registering the AID group with the system, you need to enable the AID group to receive NFC traffic. When you try to enable a non-payment AID group to receive traffic, the user is not prompted for a confirmation unless there is a conflict with one of the AIDs already registered in the system by a different app. If there is a conflict, the user will be prompted with information about which card and it's associated app will be disabled if the user chooses to enable the newly registered AID group.

Coexistence with SIM based NFC applications

In Windows 10 Mobile, the system sets up the NFC controller routing table that is used to make routing decisions at the controller layer. The table contains routing information for the following items.

- Individual AID routes.
- Protocol based route (ISO-DEP).
- Technology based routing (NFC-A/B/F).

When an external reader sends a "SELECT AID" command, the NFC controller first checks AID routes in the routing table for a match. If there is no match, it will use the protocol-based route as the default route for ISO-DEP (14443-4-A) traffic. For any other non-ISO-DEP traffic it will use the technology based routing.

Windows 10 Mobile provides a menu option "SIM Card" in the NFC Settings page to continue to use legacy Windows Phone 8.1 SIM-based apps, which do not register their AIDs with the system. If the user selects "SIM card" as their default payment card, then

the ISO-DEP route is set to UICC, for all other selections in the drop-down menu the ISO-DEP route is to the host.

The ISO-DEP route is set to "SIM Card" for devices that have an SE enabled SIM card when the device is booted for the first time with Windows 10 Mobile. When the user installs an HCE enabled app and that app enables any HCE AID group registrations, the ISO-DEP route will be pointed to the host. New SIM-based applications need to register the AIDs in the SIM in order for the specific AID routes to be populated in the controller routing table.

Creating an HCE based app

Your HCE app has two parts.

- The main foreground app for the user interaction.
- A background task that is triggered by the system to process APDUs for a given AID.

Because of the extremely tight performance requirements for loading your background task in response to an NFC tap, we recommend that your entire background task be implementing in C++/CX native code (including any dependencies, references, or libraries you depend on) rather than C# or managed code. While C# and managed code normally performs well, there is overhead, like loading the .NET CLR, that can be avoided by writing it in C++/CX.

Create and register your background task

You need to create a background task in your HCE app for processing and responding to APDUs routed to it by the system. During the first time your app is launched, the foreground registers an HCE background task that implements the [IBackgroundTaskRegistration](#) interface as shown in the following code.

C++/CX

```
var taskBuilder = new BackgroundTaskBuilder();
taskBuilder.Name = bgTaskName;
taskBuilder.TaskEntryPoint = taskEntryPoint;
taskBuilder.SetTrigger(new
SmartCardTrigger(SmartCardTriggerType.EmulatorHostApplicationActivated));
bgTask = taskBuilder.Register();
```

Notice that the task trigger is set to [SmartCardTriggerType](#).

[EmulatorHostApplicationActivated](#). This means that whenever a SELECT AID command

APDU is received by the OS resolving to your app, your background task will be launched.

Receive and respond to APDUs

When there is an APDU targeted for your app, the system will launch your background task. Your background task receives the APDU passed through the [SmartCardEmulatorApuReceivedEventArgs](#) object's [CommandApu](#) property and responds to the APDU using the [TryRespondAsync](#) method of the same object. Consider keeping your background task for light operations for performance reasons. For example, respond to the APDUs immediately and exit your background task when all processing is complete. Due to the nature of NFC transactions, users tend to hold their device against the reader for only a very short amount of time. Your background task will continue to receive traffic from the reader until your connection is deactivated, in which case you will receive a [SmartCardEmulatorConnectionDeactivatedEventArgs](#) object. Your connection can be deactivated because of the following reasons as indicated in the [SmartCardEmulatorConnectionDeactivatedEventArgs.Reason](#) property.

- If the connection is deactivated with the **ConnectionLost** value, it means that the user pulled their device away from the reader. If your app needs the user to tap to the terminal longer, you might want to consider prompting them with feedback. You should terminate your background task quickly (by completing your deferral) to ensure if they tap again it won't be delayed waiting for the previous background task to exit.
- If the connection is deactivated with the **ConnectionRedirected**, it means that the terminal sent a new SELECT AID command APDU directed to a different AID. In this case, your app should exit the background task immediately (by completing your deferral) to allow another background task to run.

The background task should also register for the [Canceled event](#) on [IBackgroundTaskInstance interface](#), and likewise quickly exit the background task (by completing your deferral) because this event is fired by the system when it is finished with your background task. Below is code that demonstrates an HCE app background task.

C++/CX

```
void BgTask::Run(
    IBackgroundTaskInstance^ taskInstance)
{
    m_triggerDetails = static_cast<SmartCardTriggerDetails^>(taskInstance->TriggerDetails);
    if (m_triggerDetails == nullptr)
```

```

{
    // May be not a smart card event that triggered us
    return;
}

m_emulator = m_triggerDetails->Emulator;
m_taskInstance = taskInstance;

switch (m_triggerDetails->TriggerType)
{
case SmartCardTriggerType::EmulatorHostApplicationActivated:
    HandleHceActivation();
    break;

case SmartCardTriggerType::EmulatorAppletIdGroupRegistrationChanged:
    HandleRegistrationChange();
    break;

default:
    break;
}
}

void BgTask::HandleHceActivation()
{
    try
    {
        auto lock = m_srwLock.LockShared();
        // Take a deferral to keep this background task alive even after
        this "Run" method returns
        // You must complete this deferral immediately after you have done
        processing the current transaction
        m_deferral = m_taskInstance->GetDeferral();

        DebugLog(L"*** HCE Activation Background Task Started ***");

        // Set up a handler for if the background task is cancelled, we must
        immediately complete our deferral
        m_taskInstance->Canceled += ref new
Windows::ApplicationModel::Background::BackgroundTaskCanceledEventHandler(
    [this](
        IBackgroundTaskInstance^ sender,
        BackgroundTaskCancellationReason reason)
    {
        DebugLog(L"Cancelled");
        DebugLog(reason.ToString()->Data());
        EndTask();
    });

        if (Windows::Phone::System::SystemProtection::ScreenLocked)
        {
            auto denyIfLocked = Windows::Storage::ApplicationData::Current-
>RoamingSettings->Values->Lookup("DenyIfPhoneLocked");
            if (denyIfLocked != nullptr && (bool)denyIfLocked == true)
            {

```

```

        // The phone is locked, and our current user setting is to
        deny transactions while locked so let the user know
        // Denied
        DoLaunch(Denied, L"Phone was locked at the time of tap");

        // We still need to respond to APDUs in a timely manner,
        even though we will just return failure
        m_fDenyTransactions = true;
    }
}
else
{
    m_fDenyTransactions = false;
}

m_emulator->ApduReceived += ref new
TypedEventHandler<SmartCardEmulator^,
SmartCardEmulatorApduReceivedEventArgs>(
    this, &BgTask::ApduReceived);

m_emulator->ConnectionDeactivated += ref new
TypedEventHandler<SmartCardEmulator^,
SmartCardEmulatorConnectionDeactivatedEventArgs>(
    [this](
        SmartCardEmulator^ emulator,
        SmartCardEmulatorConnectionDeactivatedEventArgs^ eventArgs)
    {
        DebugLog(L"Connection deactivated");
        EndTask();
    });

m_emulator->Start();
    DebugLog(L"Emulator started");
}
catch (Exception^ e)
{
    DebugLog(("Exception in Run: " + e->ToString())->Data());
    EndTask();
}
}

```

Create and register AID groups

During the first launch of your application when the card is being provisioned, you will create and register AID groups with the system. The system determines the app that an external reader would like to talk to and route APDUs accordingly based on the registered AIDs and user settings.

Most of the payment cards register for the same AID, Proximity Payment System Environment (PPSE), along with additional payment network card specific AIDs. Each AID

group represents a card and when the user enables the card, all AIDs in the group are enabled. Similarly, when the user deactivates the card, all AIDs in the group are disabled.

To register an AID group, you need to create a [SmartCardAppletIdGroup](#) object and set its properties to reflect that this is an HCE-based payment card. Your display name should be descriptive to the user because it will show up in the NFC settings menu as well as user prompts. For HCE payment cards, the [SmartCardEmulationCategory](#) property should be set to **Payment** and the [SmartCardEmulationType](#) property should be set to **Host**.

C++/CX

```
public static byte[] AID_PPSE =
{
    // File name "2PAY.SYS.DDF01" (14 bytes)
    (byte)'2', (byte)'P', (byte)'A', (byte)'Y',
    (byte)'.', (byte)'S', (byte)'Y', (byte)'S',
    (byte)'.', (byte)'D', (byte)'D', (byte)'F', (byte)'0', (byte)'1'
};

var appletIdGroup = new SmartCardAppletIdGroup(
    "Example DisplayName",
    new List<IBuffer> {AID_PPSE.AsBuffer()},
    SmartCardEmulationCategory.Payment,
    SmartCardEmulationType.Host);
```

For non-payment HCE cards, the [SmartCardEmulationCategory](#) property should be set to **Other** and the [SmartCardEmulationType](#) property should be set to **Host**.

C++/CX

```
public static byte[] AID_OTHER =
{
    (byte)'1', (byte)'2', (byte)'3', (byte)'4',
    (byte)'5', (byte)'6', (byte)'7', (byte)'8',
    (byte)'0', (byte)'T', (byte)'H', (byte)'E', (byte)'R'
};

var appletIdGroup = new SmartCardAppletIdGroup(
    "Example DisplayName",
    new List<IBuffer> {AID_OTHER.AsBuffer()},
    SmartCardEmulationCategory.Other,
    SmartCardEmulationType.Host);
```

You can include up to 9 AIDs (of length 5-16 bytes each) per AID group.

Use the [RegisterAppletIdGroupAsync](#) method to register your AID group with the system, which will return a [SmartCardAppletIdGroupRegistration](#) object. By default, the

ActivationPolicy property of the registration object is set to **Disabled**. This means even though your AIDs are registered with the system, they are not enabled yet and won't receive traffic.

C++/CX

```
reg = await SmartCardEmulator.RegisterAppletIdGroupAsync(appletIdGroup);
```

You can enable your registered cards (AID groups) by using the **RequestActivationPolicyChangeAsync** method of the **SmartCardAppletIdGroupRegistration** class as shown below. Because only a single payment card can be enabled at a time on the system, setting the **ActivationPolicy** of a payment AID group to **Enabled** is the same as setting the default payment card. The user will be prompted to allow this card as a default payment card, regardless of whether there is a default payment card already selected or not. This statement is not true if your app is already the default payment application, and is merely changing between it's own AID groups. You can register up to 10 AID groups per app.

C++/CX

```
reg.RequestActivationPolicyChangeAsync(AppletIdGroupActivationPolicy.Enabled);
```

You can query your app's registered AID groups with the OS and check their activation policy using the **GetAppletIdGroupRegistrationsAsync** method.

Users will be prompted when you change the activation policy of a payment card from **Disabled** to **Enabled**, only if your app is not already the default payment app. Users will only be prompted when you change the activation policy of a non-payment card from **Disabled** to **Enabled** if there is an AID conflict.

C++/CX

```
var registrations = await  
SmartCardEmulator.GetAppletIdGroupRegistrationsAsync();  
    foreach (var registration in registrations)  
    {  
        registration.RequestActivationPolicyChangeAsync  
        (AppletIdGroupActivationPolicy.Enabled);  
    }
```

Event notification when activation policy change

In your background task, you can register to receive events for when the activation policy of one of your AID group registrations changes outside of your app. For example, the user may change the default payment app through the NFC settings menu from one of your cards to another card hosted by another app. If your app needs to know about this change for internal setup such as updating live tiles, you can receive event notifications for this change and take action in your app accordingly.

C++/CX

```
var taskBuilder = new BackgroundTaskBuilder();
taskBuilder.Name = bgTaskName;
taskBuilder.TaskEntryPoint = taskEntryPoint;
taskBuilder.SetTrigger(new
SmartCardTrigger(SmartCardTriggerType.EmulatorAppletIdGroupRegistrationChang
ed));
bgTask = taskBuilder.Register();
```

Foreground override behavior

You can change the [ActivationPolicy](#) of any of your AID group registrations to **ForegroundOverride** while your app is in the foreground without prompting the user. When the user taps their device to a terminal while your app is in the foreground, the traffic is routed to your app even if none of your payment cards were chosen by the user as their default payment card. When you change a card's activation policy to **ForegroundOverride**, this change is only temporary until your app leaves the foreground and it will not change the current default payment card set by the user. You can change the **ActivationPolicy** of your payment or non-payment cards from your foreground app as follows. Note that the [RequestActivationPolicyChangeAsync](#) method can only be called from a foreground app and cannot be called from a background task.

C++/CX

```
reg.RequestActivationPolicyChangeAsync(AppletIdGroupActivationPolicy.Foregro
undOverride);
```

Also, you can register an AID group consisting of a single 0-length AID which will cause the system to route all APDUs regardless of the AID and including any command APDUs sent before a SELECT AID command is received. However, such an AID group only works while your app is in the foreground because it can only be set to **ForegroundOverride** and cannot be permanently enabled. Also, this mechanism works both for **Host** and **UICC** values of the [SmartCardEmulationType](#) enumeration to either route all traffic to your HCE background task, or to the SIM card.

C++/CX

```
public static byte[] AID_Foreground =  
    {};  
  
var appletIdGroup = new SmartCardAppletIdGroup(  
    "Example DisplayName",  
    new List<IBuffer>  
{AID_Foreground.AsBuffer()}},  
    SmartCardEmulationCategory.Other,  
    SmartCardEmulationType.Host);  
reg = await SmartCardEmulator.RegisterAppletIdGroupAsync(appletIdGroup);  
reg.RequestActivationPolicyChangeAsync(AppletIdGroupActivationPolicy.ForegroundOverride);
```

Check for NFC and HCE support

Your app should check whether a device has NFC hardware, supports the card emulation feature, and supports host card emulation prior to offering such features to the user.

The NFC smart card emulation feature is only enabled on Windows 10 Mobile, so trying to use the smart card emulator APIs in any other versions of Windows 10, will cause errors. You can check for smart card API support in the following code snippet.

C++/CX

```
Windows.Foundation.Metadata.ApiInformation.IsTypePresent("Windows.Devices.SmartCards.SmartCardEmulator");
```

You can additionally check to see if the device has NFC hardware capable of some form of card emulation by checking if the [SmartCardEmulator.GetDefaultAsync](#) method returns null. If it does, then no NFC card emulation is supported on the device.

C++/CX

```
var smartcardemulator = await SmartCardEmulator.GetDefaultAsync();<
```

Support for HCE and AID-based UICC routing is only available on recently launched devices such as the Lumia 730, 830, 640, and 640 XL. Any new NFC capable devices running Windows 10 Mobile and after should support HCE. Your app can check for HCE support as follows.

C++/CX


```
Smartcardemulator.IsHostCardEmulationSupported();
```

Lock screen and screen off behavior

Windows 10 Mobile has device-level card emulation settings, which can be set by the mobile operator or the manufacturer of the device. By default, "tap to pay" toggle is disabled and the "enablement policy at device level" is set to "Always", unless the MO or OEM overwrites these values.

Your application can query the value of the [EnablementPolicy](#) at device level and take action for each case depending on the desired behavior of your app in each state.

C++/CX

```
SmartCardEmulator emulator = await SmartCardEmulator.GetDefaultAsync();

switch (emulator.EnablementPolicy)
{
case Never:
    // you can take the user to the NFC settings to turn "tap and pay" on
    await Windows.System.Launcher.LaunchUriAsync(new Uri("ms-settings-nfctransactions:"));
    break;

case Always:
    return "Card emulation always on";

case ScreenOn:
    return "Card emulation on only when screen is on";

case ScreenUnlocked:
    return "Card emulation on only when screen unlocked";
}
```

Your app's background task will be launched even if the phone is locked and/or the screen is off only if the external reader selects an AID that resolves to your app. You can respond to the commands from the reader in your background task, but if you need any input from the user or if you want to show a message to the user, you can launch your foreground app with some arguments. Your background task can launch your foreground app with the following behavior.

- Under the device lock screen (the user will see your foreground app only after she unlocks the device)
- Above the device lock screen (after the user dismisses your app, the device is still in locked state)

C++/CX

```
if (Windows::Phone::System::SystemProtection::ScreenLocked)
{
    // Launch above the lock with some arguments
    var result = await eventDetails.TryLaunchSelfAsync("app-specific
arguments", SmartCardLaunchBehavior.AboveLock);
}
```

AID registration and other updates for SIM based apps

Card emulation apps that use the SIM as the secure element can register with the Windows service to declare the AIDs supported on the SIM. This registration is very similar to an HCE-based app registration. The only difference is the [SmartCardEmulationType](#), which should be set to Uicc for SIM-based apps. As the result of the payment card registration, the display name of the card will also be populated in the NFC setting menu.

C++/CX

```
var appletIdGroup = new SmartCardAppletIdGroup(
    "Example DisplayName",
    new List<IBuffer> {AID_PPSE.AsBuffer()},
    SmartCardEmulationCategory.Payment,
    SmartCardEmulationType.Uicc);
```

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Get battery information

Article • 05/06/2023

This topic describes how to get a *battery report* that includes detailed battery information (such as the charge, capacity, and status of a battery or aggregate of batteries), and handle state changes to any items in the report.

([BatteryReport](#))

Code examples are from the basic battery app that's listed at the end of this topic.

Important APIs

- [Windows.Devices.Power](#)
- [DeviceInformation.FindAllAsync](#)

Get aggregate battery report

Some devices have more than one battery and it's not always obvious how each battery contributes to the overall energy capacity of the device. This is where the [AggregateBattery](#) class comes in. The *aggregate battery* represents all battery controllers connected to the device and can provide a single overall [BatteryReport](#) object.

Note A [Battery](#) class actually corresponds to a battery controller. Depending on the device, sometimes the controller is attached to the physical battery and sometimes it's attached to the device enclosure. Thus, it's possible to create a battery object even when no batteries are present. Other times, the battery object may be **null**.

Once you have an aggregate battery object, call [GetReport](#) to get the corresponding [BatteryReport](#).

C#

```
private void RequestAggregateBatteryReport()
{
    // Create aggregate battery object
    var aggBattery = Battery.AggregateBattery;

    // Get report
    var report = aggBattery.GetReport();

    // Update UI
    AddReportUI(BatteryReportPanel, report, aggBattery.DeviceId);
}
```

Get individual battery reports

You can also create a [BatteryReport](#) object for individual batteries. Use [GetDeviceSelector](#) with the [FindAllAsync](#) method to obtain a collection of [DeviceInformation](#) objects that represent any battery controllers that are connected to the device. Then, using the `Id` property of the desired [DeviceInformation](#) object, create a corresponding [Battery](#) with the [FromIdAsync](#) method. Finally, call [GetReport](#) to get the individual battery report.

This example shows how to create a battery report for all batteries connected to the device.

```
C#

async private void RequestIndividualBatteryReports()
{
    // Find batteries
    var deviceInfo = await
DeviceInformation.FindAllAsync(Battery.GetDeviceSelector());
    foreach(DeviceInformation device in deviceInfo)
    {
        try
        {
            // Create battery object
            var battery = await Battery.FromIdAsync(device.Id);

            // Get report
            var report = battery.GetReport();

            // Update UI
            AddReportUI(BatteryReportPanel, report, battery.DeviceId);
        }
        catch { /* Add error handling, as applicable */ }
    }
}
```

Access report details

The [BatteryReport](#) object provides a lot of battery information. For more info, see the API reference for its properties: `Status` (a [BatteryStatus](#) enumeration), [ChargeRateInMilliwatts](#), [DesignCapacityInMilliwattHours](#), [FullChargeCapacityInMilliwattHours](#), and [RemainingCapacityInMilliwattHours](#). This example shows some of the battery report properties used by the basic battery app, that's provided later in this topic.

C#

```
...
TextBlock txt3 = new TextBlock { Text = "Charge rate (mW): " +
report.ChargeRateInMilliwatts.ToString() };
TextBlock txt4 = new TextBlock { Text = "Design energy capacity (mWh): " +
report.DesignCapacityInMilliwattHours.ToString() };
TextBlock txt5 = new TextBlock { Text = "Fully-charged energy capacity
(mWh): " + report.FullChargeCapacityInMilliwattHours.ToString() };
TextBlock txt6 = new TextBlock { Text = "Remaining energy capacity (mWh): "
+ report.RemainingCapacityInMilliwattHours.ToString() };
...
...
```

Request report updates

The [Battery](#) object triggers the [ReportUpdated](#) event when charge, capacity, or status of the battery changes. This typically happens immediately for status changes and periodically for all other changes. This example shows how to register for battery report updates.

C#

```
...
Battery.AggregateBattery.ReportUpdated += AggregateBattery_ReportUpdated;
...
```

Handle report updates

When a battery update occurs, the [ReportUpdated](#) event passes the corresponding [Battery](#) object to the event handler method. However, this event handler is not called from the UI thread. You'll need to use the [Dispatcher](#) object to invoke any UI changes, as shown in this example.

C#

```
async private void AggregateBattery_ReportUpdated(Battery sender, object
args)
{
    if (reportRequested)
    {

        await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, () =>
        {
            // Clear UI
            BatteryReportPanel.Children.Clear();
        });
    }
}
```

```

        if (AggregateButton.IsChecked == true)
        {
            // Request aggregate battery report
            RequestAggregateBatteryReport();
        }
        else
        {
            // Request individual battery report
            RequestIndividualBatteryReports();
        }
    });
}
}

```

Example: basic battery app

Test out these APIs by building the following basic battery app in Microsoft Visual Studio. From the Visual Studio start page, click **New Project**, and then under the **Visual C# > Windows > Universal** templates, create a new app using the **Blank App** template.

Next, open the file **MainPage.xaml** and copy the following XML into this file (replacing its original contents).

ⓘ Note

If your app isn't named **App1**, you'll need to replace the first part of the class name in the following snippet with the namespace of your app. For example, if you created a project named **BasicBatteryApp**, replace `x:Class="App1.MainPage"` with `x:Class="BasicBatteryApp.MainPage"` and `xmlns:local="using:App1"` with `xmlns:local="using:BasicBatteryApp"`.

XML

```

<Page
    x:Class="App1.MainPage"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="using:App1"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    mc:Ignorable="d">

    <StackPanel Background="{ThemeResource
        ApplicationPageBackgroundThemeBrush}" >

```

```

        <StackPanel VerticalAlignment="Center" Margin="15,30,0,0" >
            <RadioButton x:Name="AggregateButton" Content="Aggregate
results" GroupName="Type" IsChecked="True" />
            <RadioButton x:Name="IndividualButton" Content="Individual
results" GroupName="Type" IsChecked="False" />
        </StackPanel>
        <StackPanel Orientation="Horizontal">
            <Button x:Name="GetBatteryReportButton"
                Content="Get battery report"
                Margin="15,15,0,0"
                Click="GetBatteryReport"/>
        </StackPanel>
        <StackPanel x:Name="BatteryReportPanel" Margin="15,15,0,0"/>
    </StackPanel>
</Page>

```

Next, open your project's **MainPage.xaml.cs** file and replace the existing code with the following.

ⓘ Note

If your app isn't named **App1**, you'll need to rename the namespace in the following example with the name you gave your project. For example, if you created a project named **BasicBatteryApp**, you'd replace namespace **App1** with namespace **BasicBatteryApp**.

C#

```

using System;
using Windows.UI.Xaml;
using Windows.UI.Xaml.Controls;
using Windows.UI.Xaml.Media;
using Windows.Devices.Enumeration;
using Windows.Devices.Power;
using Windows.UI.Core;

namespace App1
{
    public sealed partial class MainPage : Page
    {
        bool reportRequested = false;
        public MainPage()
        {
            this.InitializeComponent();
            Battery.AggregateBattery.ReportUpdated +=
AggregateBattery_ReportUpdated;
        }
    }
}

```

```

private void GetBatteryReport(object sender, RoutedEventArgs e)
{
    // Clear UI
    BatteryReportPanel.Children.Clear();

    if (AggregateButton.IsChecked == true)
    {
        // Request aggregate battery report
        RequestAggregateBatteryReport();
    }
    else
    {
        // Request individual battery report
        RequestIndividualBatteryReports();
    }

    // Note request
    reportRequested = true;
}

private void RequestAggregateBatteryReport()
{
    // Create aggregate battery object
    var aggBattery = Battery.AggregateBattery;

    // Get report
    var report = aggBattery.GetReport();

    // Update UI
    AddReportUI(BatteryReportPanel, report, aggBattery.DeviceId);
}

async private void RequestIndividualBatteryReports()
{
    // Find batteries
    var deviceInfo = await
DeviceInformation.FindAllAsync(Battery.GetDeviceSelector());
    foreach(DeviceInformation device in deviceInfo)
    {
        try
        {
            // Create battery object
            var battery = await Battery.FromIdAsync(device.Id);

            // Get report
            var report = battery.GetReport();

            // Update UI
            AddReportUI(BatteryReportPanel, report, battery.DeviceId);
        }
        catch { /* Add error handling, as applicable */ }
    }
}

```



```

        private void AddReportUI(StackPanel sp, BatteryReport report, string
DeviceID)
        {
            // Create battery report UI
            TextBlock txt1 = new TextBlock { Text = "Device ID: " + DeviceID
};

            txt1.FontSize = 15;
            txt1.Margin = new Thickness(0, 15, 0, 0);
            txt1.TextWrapping = TextWrapping.WrapWholeWords;

            TextBlock txt2 = new TextBlock { Text = "Battery status: " +
report.Status.ToString() };
            txt2.FontStyle = Windows.UI.Text.FontStyle.Italic;
            txt2.Margin = new Thickness(0, 0, 0, 15);

            TextBlock txt3 = new TextBlock { Text = "Charge rate (mW): " +
report.ChargeRateInMilliwatts.ToString() };
            TextBlock txt4 = new TextBlock { Text = "Design energy capacity
(mWh): " + report.DesignCapacityInMilliwattHours.ToString() };
            TextBlock txt5 = new TextBlock { Text = "Fully-charged energy
capacity (mWh): " + report.FullChargeCapacityInMilliwattHours.ToString() };
            TextBlock txt6 = new TextBlock { Text = "Remaining energy
capacity (mWh): " + report.RemainingCapacityInMilliwattHours.ToString() };

            // Create energy capacity progress bar & labels
            TextBlock pbLabel = new TextBlock { Text = "Percent remaining
energy capacity" };
            pbLabel.Margin = new Thickness(0,10, 0, 5);
            pbLabel.FontFamily = new FontFamily("Segoe UI");
            pbLabel.FontSize = 11;

            ProgressBar pb = new ProgressBar();
            pb.Margin = new Thickness(0, 5, 0, 0);
            pb.Width = 200;
            pb.Height = 10;
            pb.IsIndeterminate = false;
            pb.HorizontalAlignment = HorizontalAlignment.Left;

            TextBlock pbPercent = new TextBlock();
            pbPercent.Margin = new Thickness(0, 5, 0, 10);
            pbPercent.FontFamily = new FontFamily("Segoe UI");
            pbLabel.FontSize = 11;

            // Disable progress bar if values are null
            if ((report.FullChargeCapacityInMilliwattHours == null)||
                (report.RemainingCapacityInMilliwattHours == null))
            {
                pb.IsEnabled = false;
                pbPercent.Text = "N/A";
            }
            else
            {
                pb.IsEnabled = true;
                pb.Maximum =

```

```

Convert.ToDouble(report.FullChargeCapacityInMilliwattHours);
        pb.Value =
Convert.ToDouble(report.RemainingCapacityInMilliwattHours);
        pbPercent.Text = ((pb.Value / pb.Maximum) *
100).ToString("F2") + "%";
    }

    // Add controls to stackpanel
    sp.Children.Add(txt1);
    sp.Children.Add(txt2);
    sp.Children.Add(txt3);
    sp.Children.Add(txt4);
    sp.Children.Add(txt5);
    sp.Children.Add(txt6);
    sp.Children.Add(pbLabel);
    sp.Children.Add(pb);
    sp.Children.Add(pbPercent);
}

    async private void AggregateBattery_ReportUpdated(Battery sender,
object args)
    {
        if (reportRequested)
        {

            await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, ()
=>
            {
                // Clear UI
                BatteryReportPanel.Children.Clear();

                if (AggregateButton.IsChecked == true)
                {
                    // Request aggregate battery report
                    RequestAggregateBatteryReport();
                }
                else
                {
                    // Request individual battery report
                    RequestIndividualBatteryReports();
                }
            });
        }
    }
}

```

Tip

To receive numeric values from the **BatteryReport** object, debug your app on the **Local Machine** or an external **Device**. When debugging on a device emulator, the

BatteryReport object returns **null** to the capacity and rate properties.

Customize the print workflow

Article • 05/06/2023

Create custom print workflow experiences through the use of a print workflow app.

Overview

Print workflow apps are UWP apps that expand on the functionality of [Microsoft Store devices apps \(WSDAs\)](#), so it will be helpful to have some familiarity with WSDAs before going further.

Just as in the case of WSDAs, when the user of a source application elects to print something and navigates through the print dialog, the system checks whether a workflow app is associated with that printer. If it is, the print workflow app launches (primarily as a background task; more on this below). A workflow app is able to alter both the print ticket (the XML document that configures the printer device settings for the current print task) and the actual XPS content to be printed. It can optionally expose this functionality to the user by launching a UI midway through the process. After doing its work, it passes the print content and print ticket on to the driver.

Because it involves background and foreground components, and because it is functionally coupled with other app(s), a print workflow app can be more complicated to implement than other categories of UWP apps. It is recommended that you inspect the [Workflow app sample](#)  while reading this guide to better understand how the different features can be implemented. Some features, such as various error checks and UI management, are absent from this guide for the sake of simplicity.

Getting started

The workflow app must indicate its entry point to the print system so that it can be launched at the appropriate time. This is done by inserting the following declaration in the `Application/Extensions` element of the UWP project's *package.appxmanifest* file.

XML

```
<uap:Extension Category="windows.printWorkflowBackgroundTask"
    EntryPoint="WfBackgroundTasks.WfBackgroundTask" />
```

 **Important**

There are many scenarios in which the print customization does not require user input. For this reason, Print workflow apps run as background tasks by default.

If a workflow app is associated with the source application that started the print job (see later section for instructions on this), the print system examines its manifest files for a background task entry point.

Do background work on the print ticket

The first thing the print system does with the workflow app is activate its background task (In this case, the `WfBackgroundTask` class in the `WfBackgroundTasks` namespace). In the background task's `Run` method, you should cast the task's trigger details as a `PrintWorkflowTriggerDetails` instance. This will provide the special functionality for a print workflow background task. It exposes the `PrintWorkflowSession` property, which is an instance of `PrintWorkFlowBackgroundSession`. Print workflow session classes - both the background and foreground varieties - will control the sequential steps of the print workflow app.

Then register handler methods for the two events that this session class will raise. You will define these methods later on.

C#

```
public void Run(IBackgroundTaskInstance taskInstance) {
    // Take out a deferral here and complete once all the callbacks are done
    runDeferral = taskInstance.GetDeferral();

    // Associate a cancellation handler with the background task.
    taskInstance.Canceled += new
    BackgroundTaskCanceledEventHandler(OnCanceled);

    // cast the task's trigger details as PrintWorkflowTriggerDetails
    PrintWorkflowTriggerDetails workflowTriggerDetails =
    taskInstance.TriggerDetails as PrintWorkflowTriggerDetails;

    // Get the session manager, which is unique to this print job
    PrintWorkflowBackgroundSession sessionManager =
    workflowTriggerDetails.PrintWorkflowSession;

    // add the event handler callback routines
    sessionManager.SetupRequested += OnSetupRequested;
    sessionManager.Submitted += OnXpsOMPrintSubmitted;

    // Allow the event source to start
    // This call blocks until all of the workflow callbacks complete
    sessionManager.Start();
}
```

When the `Start` method is called, the session manager will raise the **SetupRequested** event first. This event exposes general information about the print task, as well as the print ticket. At this stage, the print ticket can be edited in the background.

C#

```
private void OnSetupRequested(PrintWorkflowBackgroundSession sessionManager,
    PrintWorkflowBackgroundSetupRequestedEventArgs printTaskSetupArgs) {
    // Take out a deferral here and complete once all the callbacks are done
    Deferral setupRequestedDeferral = printTaskSetupArgs.GetDeferral();

    // Get general information about the source application, print job
    title, and session ID
    string sourceApplicationName =
    printTaskSetupArgs.Configuration.SourceAppDisplayName;
    string jobTitle = printTaskSetupArgs.Configuration.JobTitle;
    string sessionId = printTaskSetupArgs.Configuration.SessionId;

    // edit the print ticket
    WorkflowPrintTicket printTicket =
    printTaskSetupArgs.GetUserPrintTicketAsync();

    // ...
}
```

Importantly, it is in the handling of the **SetupRequested** that the app will determine whether to launch a foreground component. This could depend on a setting that was previously saved to local storage, or an event that occurred during the editing of the print ticket, or it may be a static setting of your particular app.

C#

```
// ...

if (UIRequested) {
    printTaskSetupArgs.SetRequiresUI();

    // Any data that is to be passed to the foreground task must be stored
    the app's local storage.
    // It should be prefixed with the sourceApplicationName string and the
    SessionId string, so that
    // it can be identified as pertaining to this workflow app session.
}

// Complete the deferral taken out at the start of OnSetupRequested
setupRequestedDeferral.Complete();
}
```

Do foreground work on the print job (optional)

If the [SetRequiresUI](#) method was called, then the print system will examine the manifest file for the entry point to the foreground application. The `Application/Extensions` element of your `package.appxmanifest` file must have the following lines. Replace the value of `EntryPoint` with name of the foreground app.

XML

```
<uap:Extension Category="windows.printWorkflowForegroundTask"
    EntryPoint="MyWorkFlowForegroundApp.App" />
```

Next, the print system calls the **OnActivated** method for the given app entry point. In the **OnActivated** method of its `App.xaml.cs` file, the workflow app should check the activation kind to verify that it is a workflow activation. If so, the workflow app can cast the activation arguments to a [PrintWorkflowUIActivatedEventArgs](#) object, which exposes a [PrintWorkflowForegroundSession](#) object as a property. This object, like its background counterpart in the previous section, contains events that are raised by the print system, and you can assign handlers to these. In this case, the event-handling functionality will be implemented in a separate class called `WorkflowPage`.

First, in the `App.xaml.cs` file:

C#

```
protected override void OnActivated(IActivatedEventArgs args){

    if (args.Kind == ActivationKind.PrintWorkflowForegroundTask) {

        // the app should instantiate a new UI view so that it can properly
        handle the case when
        // several print jobs are active at the same time.
        Frame rootFrame = new Frame();
        if (null == Window.Current.Content)
        {
            rootFrame.Navigate(typeof(WorkflowPage));
            Window.Current.Content = rootFrame;
        }

        // Get the main page
        WorkflowPage workflowPage = (WorkflowPage)rootFrame.Content;

        // Make sure the page knows it's handling a foreground task
        activation
        workflowPage.LaunchType =
        WorkflowPage.WorkflowPageLaunchType.ForegroundTask;

        // Get the activation arguments
```

```

        PrintWorkflowUIActivatedEventArgs printTaskUIEventArgs = args as
PrintWorkflowUIActivatedEventArgs;

        // Get the session manager
        PrintWorkflowForegroundSession taskSessionManager =
printTaskUIEventArgs.PrintWorkflowSession;

        // Add the callback handlers - these methods are in the workflowPage
class
        taskSessionManager.SetupRequested += workflowPage.OnSetupRequested;
        taskSessionManager.XpsDataAvailable +=
workflowPage.OnXpsDataAvailable;

        // start raising the print workflow events
        taskSessionManager.Start();
    }
}

```

Once the UI has attached event handlers and the **OnActivated** method has exited, the print system will fire the **SetupRequested** event for the UI to handle. This event provides the same data that the background task setup event provided, including the print job info and print ticket document, but without the ability to request the launch of additional UI. In the *WorkflowPage.xaml.cs* file:

```

C#

internal void OnSetupRequested(PrintWorkflowForegroundSession
sessionManager, PrintWorkflowForegroundSetupRequestedEventArgs
printTaskSetupArgs) {
    // If anything asynchronous is going to be done, you need to take out a
deferral here,
    // since otherwise the next callback happens once this one exits, which
may be premature
    Deferral setupRequestedDeferral = printTaskSetupArgs.GetDeferral();

    // Get information about the source application, print job title, and
session ID
    string sourceApplicationName =
printTaskSetupArgs.Configuration.SourceAppDisplayName;
    string jobTitle = printTaskSetupArgs.Configuration.JobTitle;
    string sessionId = printTaskSetupArgs.Configuration.SessionId;
    // the following string should be used when storing data that pertains
to this workflow session
    // (such as user input data that is meant to change the print content
later on)
    string localStorageVariablePrefix = string.Format("{0}::{1}::",
sourceApplicationName, sessionId);

    try
    {
        // receive and store user input
        // ...
    }
}

```



```

    }
    catch (Exception ex)
    {
        string errorMessage = ex.Message;
        Debug.WriteLine(errorMessage);
    }
    finally
    {
        // Complete the deferral taken out at the start of OnSetupRequested
        setupRequestedDeferral.Complete();
    }
}

```

Next, the print system will raise the **XpsDataAvailable** event for the UI. In the handler for this event, the workflow app can access all of the data available to the setup event and can additionally read the XPS data directly, either as a stream of raw bytes or as an object model. Access to the XPS data allows the UI to provide print preview services and to provide additional information to the user about the operations that the workflow app will execute on the data.

As part of this event handler, the workflow app must acquire a deferral object if it will continue to interact with the user. Without a deferral, the print system will consider the UI task complete when the **XpsDataAvailable** event handler exits or when it calls an async method. When the app has gathered all required information from the user's interaction with the UI, it should complete the deferral so that the print system can then advance.

C#

```

internal async void OnXpsDataAvailable(PrintWorkflowForegroundSession
sessionManager, PrintWorkflowXpsDataAvailableEventArgs
printTaskXpsAvailableEventArgs)
{
    // Take out a deferral
    Deferral xpsDataAvailableDeferral =
    printTaskXpsAvailableEventArgs.GetDeferral();

    SpoolStreamContent xpsStream = printTaskXpsAvailableEventArgs.Operation.XpsC
ontent.GetSourceSpoolDataAsStreamContent();

    IInputStream inputStream = xpsStream.GetInputSpoolStream();

    using (var inputReader = new Windows.Storage.Streams.DataReader(inputStr
eam))
    {
        // Read the XPS data from input stream
        byte[] xpsData = new byte[inputReader.UnconsumedBufferLength];
        while (inputReader.UnconsumedBufferLength > 0)
        {

```

```

        inputReader.ReadBytes(xpsData);
        // Do something with the XPS data, e.g. preview
        // ...
    }
}

// Complete the deferral taken out at the start of this method
xpsDataAvailableDeferral.Complete();
}

```

Additionally, the [PrintWorkflowSubmittedOperation](#) instance exposed by the event args provides the option to cancel the print job or to indicate that the job is successful but that no output print job will be needed. This is done by calling the [Complete](#) method with a [PrintWorkflowSubmittedStatus](#) value.

ⓘ Note

If the workflow app cancels the print job, it is highly recommended that it provide a toast notification indicating why the job was cancelled.

Do final background work on the print content

Once the UI has completed the deferral in the **PrintTaskXpsDataAvailable** event (or if the UI step was bypassed), the print system will fire the [Submitted](#) event for the background task. In the handler for this event, the workflow app can get access to all of the same data provided by the **XpsDataAvailable** event. However, unlike any of the previous events, **Submitted** also provides *write* access to the final print job content through a [PrintWorkflowTarget](#) instance.

The object that is used to spool the data for final printing depends on whether the source data is accessed as a raw byte stream or as the XPS object model. When the workflow app accesses the source data through a byte stream, an output byte stream is provided to write the final job data to. When the workflow app accesses the source data through the object model, a document writer is provided to write objects to the output job. In either case, the workflow app should read all of the source data, modify any data required, and write the modified data to the output target.

When the background task finishes writing the data, it should call **Complete** on the corresponding **PrintWorkflowSubmittedOperation** object. Once the workflow app completes this step and the **Submitted** event handler exits, the workflow session is closed and the user can monitor the status of the final print job through the standard print dialogs.

Final steps

Register the print workflow app to the printer

Your workflow app is associated with a printer using the same type of metadata file submission as for WSDAs. In fact, a single UWP application can act as both a workflow app and a WSDA that provides print task settings functionality. Follow the corresponding [WSDA steps for creating the metadata association](#).

The difference is that while WSDAs are automatically activated for the user (the app will always launch when that user prints on the associated device), workflow apps are not. They have a separate policy that must be set.

Set the workflow app's policy

The workflow app policy is set by Powershell commands on the device that is to run the workflow app. The Set-Printer, Add-Printer (existing port) and Add-Printer (new WSD port) commands will be modified to allow Workflow policies to be set.

- **Disabled**: Workflow apps will not be activated.
- **Uninitialized**: Workflow apps will be activated if the Workflow DCA is installed in the system. If the app is not installed, printing will still proceed.
- **Enabled**: Workflow contract will be activated if the Workflow DCA is installed in the system. If the app is not installed, printing will fail.

The following command makes the workflow app required on the specified printer.

Powershell

```
Set-Printer -Name "Microsoft XPS Document Writer" -WorkflowPolicy Enabled
```

A local user can run this policy on a local printer, or, for enterprise implementation, the printer administrator can run this policy on the Print Server. The policy will then be synchronized to all client connections. The printer admin can use this policy whenever a new printer is added.

See also

[Workflow app sample](#) ↗

[Windows.Graphics.Printing.Workflow namespace](#)

Launching, resuming, and background tasks

Article • 10/20/2022

This section includes information on the following:

- What happens when a Universal Windows Platform (UWP) app is started, suspended, resumed, and terminated.
- How to launch apps by using a URI or by file activation.
- How to use App services, which allow your Universal Windows Platform (UWP) app to share data and functionality with other apps.
- How to use background tasks, which allow a UWP app to do work while the app itself is not in the foreground.
- How to discover connected devices, launch an app on another device, and communicate with an app service on a remote device so that you can create user experiences that flow across devices.
- How to choose the right technology to extend and componentize your app.
- How to add and configure a splash screen for your app.
- How to write extend your app via packages from that users can install from the Microsoft Store.

The app lifecycle

This section details the lifecycle of a Windows 10 Universal Windows Platform (UWP) app, from the time it is activated until it is closed.

Topic	Description
App lifecycle	Learn about the life cycle of a UWP app and what happens when Windows launches, suspends, and resumes your app.
Handle app prelaunch	Learn how to handle app prelaunch.
Handle app activation	Learn how to handle app activation.
Handle app suspend	Learn how to save important application data when the system suspends your app.
Handle app resume	Learn how to refresh displayed content when the system resumes your app.

Topic	Description
Free memory when your app moves to the background	Learn how to reduce the amount of memory that your app uses when it is in the background state so that it won't be terminated.
Postpone app suspension with extended execution	Learn how to use extended execution to keep your app running when it is minimized

Launch apps

Topic	Description
Create a Universal Windows Platform console app	Learn how to write a Universal Windows Platform app that runs in a console window.
Create a Multi-instance UWP app	Learn how to write a multi-instance Universal Windows Platform app.

The [Launch an app with a URI](#) section details how to use a Uniform Resource Identifier (URI) to launch an app.

Topic	Description
Launch the default app for a URI	Learn how to launch the default app for a Uniform Resource Identifier (URI). URIs allow you to launch another app to perform a specific task. This topic also provides an overview of the many URI schemes built into Windows.
Handle URI activation	Learn how to register an app to become the default handler for a Uniform Resource Identifier (URI) scheme name.
Launch an app for results	Learn how to launch an app from another app and exchange data between the two. This is called launching an app for results.
Choose and save tones using the ms-tonepicker URI scheme	This topic describes the ms-tonepicker URI scheme and how to use it to display a tone picker to select a tone, save a tone, and get the friendly name for a tone.
Launch the Windows Settings app	Learn how to launch the Windows Settings app from your app. This topic describes the ms-settings URI scheme. Use this URI scheme to launch the Windows Settings app to specific settings pages.
Launch the Microsoft Store app	This topic describes the ms-windows-store URI scheme. Your app can use this URI scheme to launch the UWP app to specific pages in the Store.

Topic	Description
Launch the Windows Maps app	Learn how to launch the Windows Maps app from your app.
Launch the People app	This topic describes the ms-people URI scheme. Your app can use this URI scheme to launch the People app for specific actions.
Support web-to-app linking with app URI handlers	Drive user engagement with your app by using app URI handlers.

The [Launch an app through file activation](#) section details how to set up your app to launch when a file of a certain type is opened.

Topic	Description
Launch the default app for a file	Learn how to launch the default app for a file.
Handle file activation	Learn how to register your app to become the default handler for a certain file type.

See other topics related to launching an app below.

Topic	Description
Continue user activity, even across devices	Reengage users with your app, even across devices, by launching your app where the user left off.
Auto-launching with AutoPlay	You can use AutoPlay to provide your app as an option when a user connects a device to their PC. This includes non-volume devices such as a camera or media player, or volume devices such as a USB thumb drive, SD card, or DVD.
Reserved file and URI scheme names	This topic lists the reserved file and URI scheme names that are not available to your app.

App services and extensions

The [App services and extensions](#) section describes how to integrate app services into your UWP app to allow the sharing of data and functionality across apps.

Topic	Description
Create and consume an app service	Learn how to write a Universal Windows Platform (UWP) app that can provide services to other UWP apps and how to consume those services.
Convert an app service to run in the same process as its host app	Convert app service code that ran in a separate background process into code that runs inside the same process as your app service provider.
Extend your app with app services, extensions, and packages	Determine which technology to use to extend and componentize your app and get a brief overview of each.
Create and consume an app extension	Write and host Universal Windows Platform (UWP) app extensions to extend your app via packages that users can install from the Microsoft Store.

Background tasks

The [Background tasks](#) section shows you how to make lightweight code run in the background in response to triggers.

Topic	Description
Guidelines for background tasks	Ensure your app meets the requirements for running background tasks.
Access sensors and devices from a background task	DeviceUseTrigger lets your Universal Windows app access sensors and peripheral devices in the background, even when your foreground app is suspended.
Create and register an in-process background task	Create and register a background task that runs in the same process as your foreground app.
Create and register an out-of-process background task	Create and register a background task that runs in a separate process from your app, and register it to run when your app is not in the foreground.
Create and register a COM background task for a Win32 app	Create a COM background task that can run in your main process or out-of-process when your packaged Win32 app may not be running.
Port an out-of-process background task to an in-process background task	Learn how to port an out-of-process background task to an in-process background task that runs in the same process as your foreground app.
Debug a background task	Learn how to debug a background task, including background task activation and debug tracing in the Windows event log.

Topic	Description
Declare background tasks in the application manifest	Enable the use of background tasks by declaring them as extensions in the app manifest.
Group background task registration	Isolate background task registration with groups.
Handle a cancelled background task	Learn how to make a background task that recognizes cancellation requests and stops work, reporting the cancellation to the app using persistent storage.
Monitor background task progress and completion	Learn how your app can recognize background task progress and completion.
Optimize background activity	Learn how to reduce the energy used in the background and interact with user settings for background activity.
Register a background task	Learn how to create a function that can be re-used to safely register most background tasks.
Respond to system events with background tasks	Learn how to create a background task that responds to SystemTrigger events.
Run a background task on a timer	Learn how to schedule a one-time background task, or run a periodic background task.
Run in the background indefinitely	Use a capability to run a background task or extended execution session in the background indefinitely.
Trigger a background task from within your app	Learn how to use the ApplicationTrigger to activate a background task from within your app.
Set conditions for running a background task	Learn how to set conditions that control when your background task will run.
Transfer data in the background	Use the background transfer API to copy files in the background.
Update a live tile from a background task	Use a background task to update your app's live tile with fresh content.
Use a maintenance trigger	Learn how to use the MaintenanceTrigger class to run lightweight code in the background while the device is plugged in.

Remote Systems

The [Connected apps and devices \(Project Rome\)](#) section describes how to use the Remote Systems platform to discover remote devices, launch an app on a remote

device, and communicate with an app service on a remote device.

Topic	Description
Discover remote devices	Learn how to discover devices that you can connect to.
Launch an app on a remote device	Learn how to launch an app on a remote device.
Communicate with a remote app service	Learn how to interact with an app on a remote device.
Connect devices through remote sessions	Create shared experiences across multiple devices by joining them in a remote session.

Splash screens

The [Splash screens](#) section describes how to set and configure your app's splash screen.

Topic	Description
Add a splash screen	Set your app's splash screen image and background color.
Display a splash screen for more time	Display a splash screen for more time by creating an extended splash screen for your app. This extended screen imitates the splash screen shown when your app is launched, and can be customized.

Create a Universal Windows Platform console app

Article • 10/20/2022

This topic describes how to create a [C++/WinRT](#) or C++/CX Universal Windows Platform (UWP) console app.

ⓘ Note

The **Console App (Universal) Project Templates** [extension](#) does not support Visual Studio 2022. Either Visual Studio 2017 or Visual Studio 2019 is required to install and use these templates.

Starting with Windows 10, version 1803, you can write C++/WinRT or C++/CX UWP console apps that run in a console window, such as a DOS or PowerShell console window. Console apps use the console window for input and output, and can use [Universal C Runtime](#) functions such as `printf` and `getchar`. UWP console apps can be published to the Microsoft Store. They have an entry in the app list, and a primary tile that can be pinned to the Start menu. UWP console apps can be launched from the Start menu, though you will typically launch them from the command-line.

To see one in action, here's a video about Creating a UWP Console App.

<https://www.youtube-nocookie.com/embed/StmP7hYk1-w> [↗](#)

Use a UWP Console app template

To create a UWP console app, first install the **Console App (Universal) Project Templates**, available from the [Visual Studio Marketplace](#) [↗](#). The installed templates are then available under **New Project > Installed > Other Languages > Visual C++ > Windows Universal as Console App C++/WinRT (Universal Windows)** and **Console App C++/CX (Universal Windows)**.

Add your code to main()

The templates add **Program.cpp**, which contains the `main()` function. This is where execution begins in a UWP console app. Access the command-line arguments with the `__argc` and `__argv` parameters. The UWP console app exits when control returns from `main()`.

The following example of **Program.cpp** is added by the **Console App C++/WinRT** template:

C++/WinRT

```
#include "pch.h"

using namespace winrt;

// This example code shows how you could implement the required main
// function
// for a Console UWP Application. You can replace all the code inside main
// with your own custom code.

int __cdecl main()
{
    // You can get parsed command-line arguments from the CRT globals.
    wprintf(L"Parsed command-line arguments:\n");
    for (int i = 0; i < __argc; i++)
    {
        wprintf(L"__argv[%d] = %S\n", i, __argv[i]);
    }

    // Keep the console window alive in case you want to see console output
    // when running from within Visual Studio
    wprintf(L"Press 'Enter' to continue: ");
    getchar();
}
```

UWP Console app behavior

A UWP Console app can access the file-system from the directory it is run from, and below. This is possible because the template adds the [AppExecutionAlias](#) extension to your app's Package.appxmanifest file. This extension also enables the user to type the alias from a console window to launch the app. The app does not need to be in the system path to launch.

You can additionally give broad access to the file system to your UWP console app by adding the restricted capability `broadFileSystemAccess` as described in [File access permissions](#). This capability works with APIs in the [Windows.Storage](#) namespace.

More than one instance of a UWP Console app can run at a time because the template adds the [SupportsMultipleInstances](#) capability to your app's Package.appxmanifest file.

The template also adds the `Subsystem="console"` capability to the Package.appxmanifest file, which denotes that this UWP app is a console app. Note the `desktop4` and `iot2`

namespace prefixes. UWP Console apps are only supported on desktop and Internet of Things (IoT) projects.

XML

```
<Package
  ...

  xmlns:desktop4="http://schemas.microsoft.com/appx/manifest/desktop/windows10/4"
  xmlns:iot2="http://schemas.microsoft.com/appx/manifest/iot/windows10/2"
  IgnorableNamespaces="uap mp uap5 desktop4 iot2">
  ...
  <Applications>
    <Application Id="App"
      ...
      desktop4:Subsystem="console"
      desktop4:SupportsMultipleInstances="true"
      iot2:Subsystem="console"
      iot2:SupportsMultipleInstances="true" >
      ...
      <Extensions>
        <uap5:Extension
          Category="windows.appExecutionAlias"
          Executable="YourApp.exe"
          EntryPoint="YourApp.App">
            <uap5:AppExecutionAlias desktop4:Subsystem="console">
              <uap5:ExecutionAlias Alias="YourApp.exe" />
            </uap5:AppExecutionAlias>
          </uap5:Extension>
        </Extensions>
      </Application>
    </Applications>
    ...
  </Package>
```

Additional considerations for UWP console apps

- Only C++/WinRT and C++/CX UWP apps may be console apps.
- UWP Console apps must target the Desktop, or IoT project type.
- UWP console apps may not create a window. They cannot use `MessageBox()`, or `Location()`, or any other API that may create a window for any reason, such as user consent prompts.
- UWP console apps may not consume background tasks nor serve as a background task.

- With the exception of [Command-Line activation](#), UWP console apps do not support activation contracts, including file association, protocol association, etc.
- Although UWP console apps support multi-instancing, they do not support [Multi-instancing redirection](#)
- For a list of Win32 APIs that are available to UWP apps, see [Win32 and COM APIs for UWP apps](#)

Create a multi-instance Universal Windows App

Article • 02/29/2024

This topic describes how to create multi-instance Universal Windows Platform (UWP) apps.

From Windows 10, version 1803 (10.0; Build 17134) onward, your UWP app can opt in to support multiple instances. If an instance of an multi-instance UWP app is running, and a subsequent activation request comes through, the platform will not activate the existing instance. Instead, it will create a new instance, running in a separate process.

Important

Multi-instancing is supported for JavaScript applications, but multi-instancing redirection is not. Since multi-instancing redirection is not supported for JavaScript applications, the [AppInstance](#) class is not useful for such applications.

Opt in to multi-instance behavior

If you are creating a new multi-instance application, you can install the [Multi-Instance App Project Templates.VSIX](#), available from the [Visual Studio Marketplace](#). Once you install the templates, they will be available in the **New Project** dialog under **Visual C# > Windows Universal** (or **Other Languages > Visual C++ > Windows Universal**).

Note

The Multi-Instance App Project template is no longer available. The VSIX template was a convenience, so you will need to modify the existing project instead, as described below. Be certain to add the `DISABLE_XAML_GENERATED_MAIN` constant to the project build symbols, as this prevents the build from generating a default `Main()`. This allows used of a specially written app-specific version of `Main()`.

Two templates are installed: **Multi-Instance UWP app**, which provides the template for creating a multi-instance app, and **Multi-Instance Redirection UWP app**, which provides additional logic that you can build on to either launch a new instance or selectively activate an instance that has already been launched. For example, perhaps you only

want one instance at a time editing the same document, so you bring the instance that has that file open to the foreground rather than launching a new instance.

Both templates add `SupportsMultipleInstances` to the `package.appxmanifest` file. Note the namespace prefix `desktop4` and `iot2`: only projects that target the desktop, or Internet of Things (IoT) projects, support multi-instancing.

XML

```
<Package
...

xmlns:desktop4="http://schemas.microsoft.com/appx/manifest/desktop/windows10/4"
xmlns:iot2="http://schemas.microsoft.com/appx/manifest/iot/windows10/2"
IgnorableNamespaces="uap mp desktop4 iot2">
...
<Applications>
  <Application Id="App"
    ...
    desktop4:SupportsMultipleInstances="true"
    iot2:SupportsMultipleInstances="true">
    ...
  </Application>
</Applications>
...
</Package>
```

Multi-instance activation redirection

Multi-instancing support for UWP apps goes beyond simply making it possible to launch multiple instances of the app. It allows for customization in cases where you want to select whether a new instance of your app is launched or an instance that is already running is activated. For example, if the app is launched to edit a file that is already being edited in another instance, you may want to redirect the activation to that instance instead of opening up another instance that that is already editing the file.

To see it in action, watch this video about Creating multi-instance UWP apps.

<https://www.youtube-nocookie.com/embed/0oZ2vrc1-fc> ↗

The **Multi-Instance Redirection UWP app** template adds `SupportsMultipleInstances` to the `package.appxmanifest` file as shown above, and also adds a **Program.cs** (or **Program.cpp**, if you are using the C++ version of the template) to your project that contains a `Main()` function. The logic for redirecting activation goes in the `Main` function. The template for **Program.cs** is shown below.

The [AppInstance.RecommendedInstance](#) property represents the shell-provided preferred instance for this activation request, if there is one (or `null` if there isn't one). If the shell provides a preference, then you can redirect activation to that instance, or you can ignore it if you choose.

C#

```
public static class Program
{
    // This example code shows how you could implement the required Main
    // method to
    // support multi-instance redirection. The minimum requirement is to
    // call
    // Application.Start with a new App object. Beyond that, you may delete
    // the
    // rest of the example code and replace it with your custom code if you
    // wish.

    static void Main(string[] args)
    {
        // First, we'll get our activation event args, which are typically
        // richer
        // than the incoming command-line args. We can use these in our app-
        // defined
        // logic for generating the key for this instance.
        IActivatedEventArgs activatedArgs =
        AppInstance.GetActivatedEventArgs();

        // If the Windows shell indicates a recommended instance, then
        // the app can choose to redirect this activation to that instance
        // instead.
        if (AppInstance.RecommendedInstance != null)
        {
            AppInstance.RecommendedInstance.RedirectActivationTo();
        }
        else
        {
            // Define a key for this instance, based on some app-specific
            // logic.
            // If the key is always unique, then the app will never
            // redirect.
            // If the key is always non-unique, then the app will always
            // redirect
            // to the first instance. In practice, the app should produce a
            // key
            // that is sometimes unique and sometimes not, depending on its
            // own needs.
            string key = Guid.NewGuid().ToString(); // always unique.
                                                    //string key = "Some-
            App-Defined-Key"; // never unique.
            var instance = AppInstance.FindOrRegisterInstanceForKey(key);
            if (instance.IsCurrentInstance)
            {

```



```

just
    // If we successfully registered this instance, we can now
    // go ahead and do normal XAML initialization.
    global::Windows.UI.Xaml.Application.Start((p) => new App());
}
else
{
    // Some other instance has registered for this key, so we'll
    // redirect this activation to that instance instead.
    instance.RedirectActivationTo();
}
}
}
}
}

```

`Main()` is the first thing that runs. It runs before [OnLaunched](#) and [OnActivated](#). This allows you to determine whether to activate this, or another instance, before any other initialization code in your app runs.

The code above determines whether an existing, or new, instance of your application is activated. A key is used to determine whether there is an existing instance that you want to activate. For example, if your app can be launched to [Handle file activation](#), you might use the file name as a key. Then you can check whether an instance of your app is already registered with that key and activate it instead of opening a new instance. This is the idea behind the code: `var instance =`

```
AppInstance.FindOrRegisterInstanceForKey(key);
```

If an instance registered with the key is found, then that instance is activated. If the key is not found, then the current instance (the instance that is currently running `Main`) creates its application object and starts running.

Background tasks and multi-instancing

- Out-of-proc background tasks support multi-instancing. Typically, each new trigger results in a new instance of the background task (although technically speaking multiple background tasks may run in same host process). Nevertheless, a different instance of the background task is created.
- In-proc background tasks do not support multi-instancing.
- Background audio tasks do not support multi-instancing.
- When an app registers a background task, it usually first checks to see if the task is already registered and then either deletes and re-registers it, or does nothing in order to keep the existing registration. This is still the typical behavior with multi-instance apps. However, a multi-instancing app may choose to register a different background task name on a per-instance basis. This will result in multiple

registrations for the same trigger, and multiple background task instances will be activated when the trigger fires.

- App-services launch a separate instance of the app-service background task for every connection. This remains unchanged for multi-instance apps, that is each instance of a multi-instance app will get its own instance of the app-service background task.

Additional considerations

- Multi-instancing is supported by UWP apps that target desktop and Internet of Things (IoT) projects.
- To avoid race-conditions and contention issues, multi-instance apps need to take steps to partition/synchronize access to settings, app-local storage, and any other resource (such as user files, a data store, and so on) that can be shared among multiple instances. Standard synchronization mechanisms such as mutexes, semaphores, events, and so on, are available.
- If the app has `SupportsMultipleInstances` in its Package.appxmanifest file, then its extensions do not need to declare `SupportsMultipleInstances`.
- If you add `SupportsMultipleInstances` to any other extension, apart from background tasks or app-services, and the app that hosts the extension doesn't also declare `SupportsMultipleInstances` in its Package.appxmanifest file, then a schema error is generated.
- Apps can use the `ResourceGroup` declaration in their manifest to group multiple background tasks into the same host. This conflicts with multi-instancing, where each activation goes into a separate host. Therefore an app cannot declare both `SupportsMultipleInstances` and `ResourceGroup` in their manifest.

Sample

See [Multi-Instance sample](#)  for an example of multi-instance activation redirection.

See also

[AppInstance.FindOrRegisterInstanceForKey](#) [AppInstance.GetActivatedEventArgs](#)
[AppInstance.RedirectActivationTo](#) Handle app activation

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#) 

| [Get help at Microsoft Q&A](#)

Universal Windows Platform (UWP) app lifecycle

Article • 09/13/2023

This topic describes the lifecycle of a Universal Windows Platform (UWP) app from the time it is launched until it is closed.

A little history

Before Windows 8, apps had a simple lifecycle. Win32 and .NET apps are either running or not running. When a user minimizes them, or switches away from them, they continue to run. This was fine until portable devices and power management became increasingly important.

Windows 8 introduced a new application model with UWP apps. At a high level, a new suspended state was added. A UWP app is suspended shortly after the user minimizes it or switches to another app. This means that the app's threads are stopped and the app is left in memory unless the operating system needs to reclaim resources. When the user switches back to the app, it can be quickly restored to a running state.

There are various ways for apps that need to continue to run when they are in the background such as [background tasks](#), [extended execution](#), and activity sponsored execution (for example, the **BackgroundMediaEnabled** capability which allows an app to continue to [play media in the background](#)). Also, background transfer operations can continue even if your app is suspended or even terminated. For more info, see [How to download a file](#).

By default, apps that are not in the foreground are suspended. This results in power savings and more resources available for the app currently in the foreground.

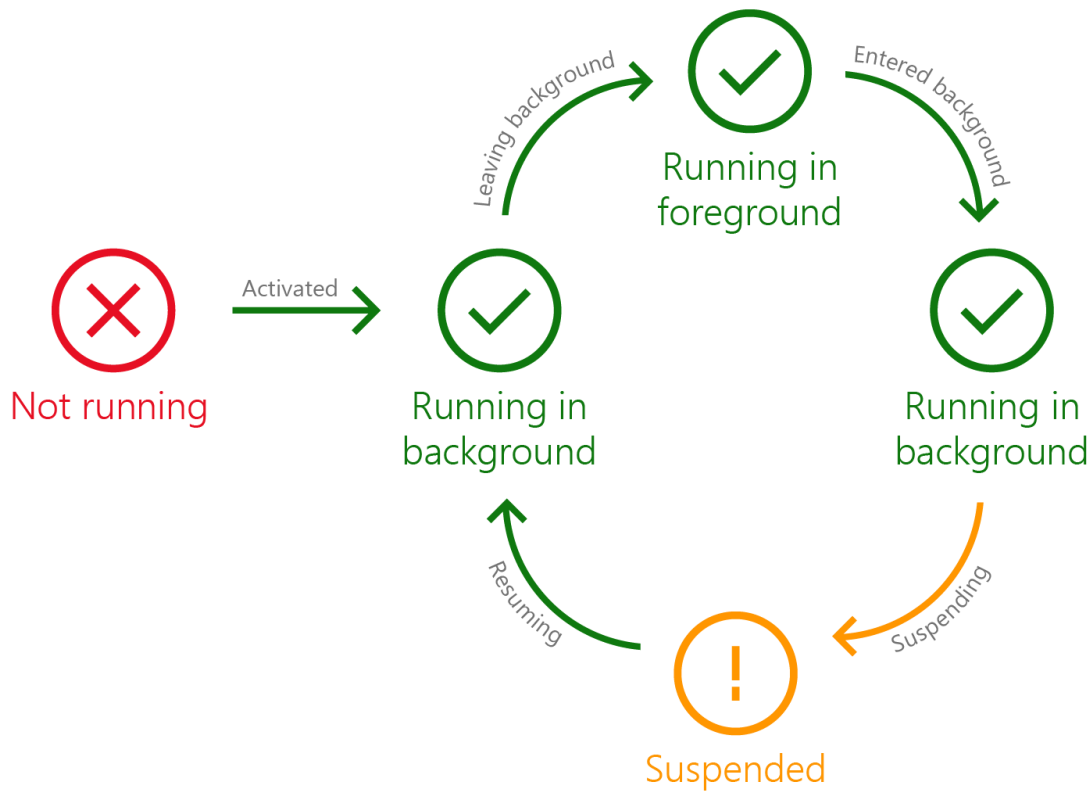
The suspended state adds new requirements for you as a developer because the operating system may elect to terminate a suspended app in order to free up resources. The terminated app will still appear in the task bar. When the user click on it, the app must restore the state that it was in before it was terminated because the user will not be aware that the system closed the app. They will think that it has been waiting in the background while they were doing other things and will expect it to be in the same state it was in when they left it. In this topic we will look at how to accomplish that.

Windows 10, version 1607, introduced two more app model states: **Running in foreground** and **Running in background**. We will look at these additional states in the

sections that follow.

App execution state

This illustration represents the possible app model states starting in Windows 10, version 1607. Let's walk through the typical lifecycle of a UWP app.



Apps enter the running in background state when they are launched or activated. If the app needs to move into the foreground due to a foreground app launch, the app then gets the [LeavingBackground](#) event.

Although "launched" and "activated" may seem like similar terms, they refer to different ways the operating system may start your app. Let's first look at launching an app.

App launch

The [OnLaunched](#) method is called when an app is launched. It is passed a [LaunchActivatedEventArgs](#) parameter which provides, among other things, the arguments passed to the app, the identifier of the tile that launched the app, and the previous state that the app was in.

Get the previous state of your app from

[LaunchActivatedEventArgs.PreviousExecutionState](#) which returns an